

ส่วนที่ 1: หลักการออกแบบ (Design Principles)

1.1 การประยุกต์ใช้ SOLID Principles กับการออกแบบ Entity และสถาปัตยกรรม

หลักการออกแบบ SOLID ถูกนำมาใช้ในการพัฒนาระบบทั้งในระดับโครงสร้างของ Entity (Database Modeling) และระดับสถาปัตยกรรมซอฟต์แวร์ (Layered Architecture) ดังนี้:

| ตัวย่อ | หลักการ (Principle)                   | ความหมายและการประยุกต์ใช้กับ Entity และระบบใน Lab 8                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| S      | Single Responsibility Principle (SRP) | "หนึ่งคลาส/หนึ่งตาราง ควรมีหน้าที่และเหตุผลในการเปลี่ยนแปลงเพียงเรื่องเดียว"<br><ul style="list-style-type: none"><li>Entity Level: แยกข้อมูลสินค้าหลัก (Product) ออกจากรายละเอียดเชิงลึก (ProductDetail) และข้อมูลความคิดเห็นลูกค้า (Review) แทนที่จะรวมทุกคอลัมน์ไว้ในตารางเดียว (หลีกเลี่ยง God Table)</li><li>Layered Architecture: แยกหน้าที่ชัดเจนระหว่าง ProductController (จัดการ HTTP/Routing), ProductService (Business Logic &amp; Transactions) และ ProductRepository (Persistence)</li></ul>                          |
| O      | Open/Closed Principle (OCP)           | "เปิดรับการต่อขยาย (Open for Extension) แต่ปิดรับการแก้ไขของเดิม (Closed for Modification)"<br><ul style="list-style-type: none"><li>Entity Level: เมื่อต้องการเพิ่มระบบรีวิวลินค้า เราสร้าง Entity Review เป็นตารางใหม่ที่มีความสัมพันธ์ 1:N โดยไม่ต้องแก้ไขหรือเปลี่ยนโครงสร้างของตาราง products เดิม</li><li>Strategy Pattern: สามารถเพิ่มประเภทส่วนลดใหม่ (เช่น VipDiscountStrategy) ได้ง่าย เพียง implement อินเทอร์เฟซ DiscountStrategy เพิ่มเติม โดยไม่ต้องแก้ไขโค้ดใน DiscountContext หรือ ProductService</li></ul>        |
| L      | Liskov Substitution Principle (LSP)   | "คลาสลูกหรือคลาสที่ Implement Interface ต้องสามารถใช้งานแทนที่ Interface นั้นได้โดยไม่ทำให้ระบบทำงานผิดพลาด"<br><ul style="list-style-type: none"><li>Repository Level: ProductRepository ขยายมาจาก JpaRepository&lt;Product, Long&gt; สามารถถูกใช้งานแทน Interface มาตรฐานของ Spring Data JPA ได้ทุก method เช่น findAll(), save(), deleteById()</li><li>Strategy Level: ทุกกลยุทธ์ส่วนลด (NoDiscountStrategy, MemberDiscountStrategy, SeasonalSaleStrategy) สามารถทำงานทดแทนกันผ่าน DiscountStrategy ได้อย่างไร้รอยต่อ</li></ul> |
| I      | Interface Segregation Principle (ISP) | "ไม่ควรบังคับให้ Client ขึ้นกับ Methods หรือ Interface ที่ตนไม่ได้ใช้งาน"<br><ul style="list-style-type: none"><li>Repository Level: มีการแยก Interface จัดการข้อมูลตามแต่ละ Entity เช่น ProductRepository, ProductDetailRepository, ReviewRepository แทนที่จะรวมเป็น Repository ขนาดยักษ์ตัวเดียว</li><li>Strategy Level: DiscountStrategy ออกแบบให้มีขนาดเล็กและเฉพาะเจาะจง มีเพียง 2 methods ที่จำเป็นคือ applyDiscount(price) และ getDiscountType()</li></ul>                                                                  |
| D      | Dependency Inversion Principle (DIP)  | "High-level module ต้องไม่ขึ้นกับ Low-level module แต่ทั้งสองต้องขึ้นกับ Abstraction"<br><ul style="list-style-type: none"><li>ProductService ไม่ได้ขึ้นกับคลาส Database โดยตรง แต่ขึ้นกับ Abstraction คือ Interface ProductRepository</li><li>ProductController และ ProductService ใช้ Constructor Injection ในการรับ Dependencies เข้ามา ทำให้ Spring Framework ควบคุมการฉีด Object (Inversion of Control - IoC)</li></ul>                                                                                                       |

## 1.2 ความแตกต่างระหว่างความสัมพันธ์ 1:1 กับ 1:N และเกณฑ์การเลือกใช้งาน

ในการออกแบบฐานข้อมูลเชิงสัมพันธ์ (Relational Database) ด้วย Spring Data JPA ความสัมพันธ์ทั้งสองแบบมีโครงสร้างและการใช้งานที่แตกต่างกันอย่างชัดเจน:



## 1.3 การประยุกต์ใช้ Strategy Pattern สำหรับคำนวณส่วนลด

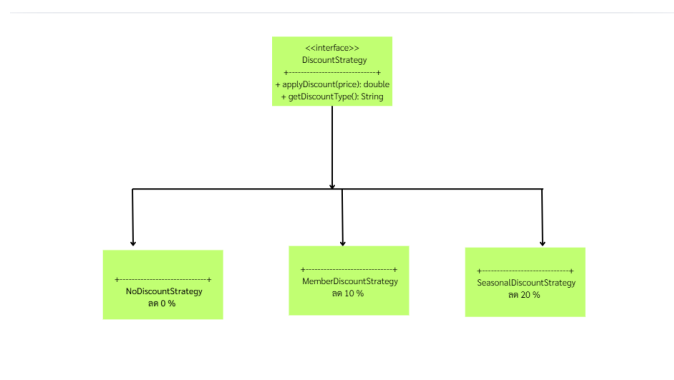
ปัญหาที่พบในการเขียนโปรแกรมแบบเดิม

หากไม่มีการใช้ Design Pattern โค้ดคำนวณส่วนลดมักจะใช้ if-else หรือ switch-case ซ้อนกันอยู่ใน Service Layer:

```
// โค้ดแบบเดิมที่ฝ่าฝืน OCP:
if ("MEMBER".equals(discountType)) {
    price = price * 0.90;
} else if ("SEASONAL".equals(discountType)) {
    price = price * 0.80;
}
// หากมีโปรโมชั่นใหม่ ต้องกลับมาแก้โค้ดเดิม เสี่ยงเกิด Bug
```

การแก้ปัญหาด้วย Strategy Pattern

Strategy Pattern ช่วยแยกอัลกอริทึมการคำนวณออกเป็นคลาสเดี่ยว ๆ ที่สามารถสับเปลี่ยน (Interchangeable) ได้ในขณะทำงาน (Runtime):



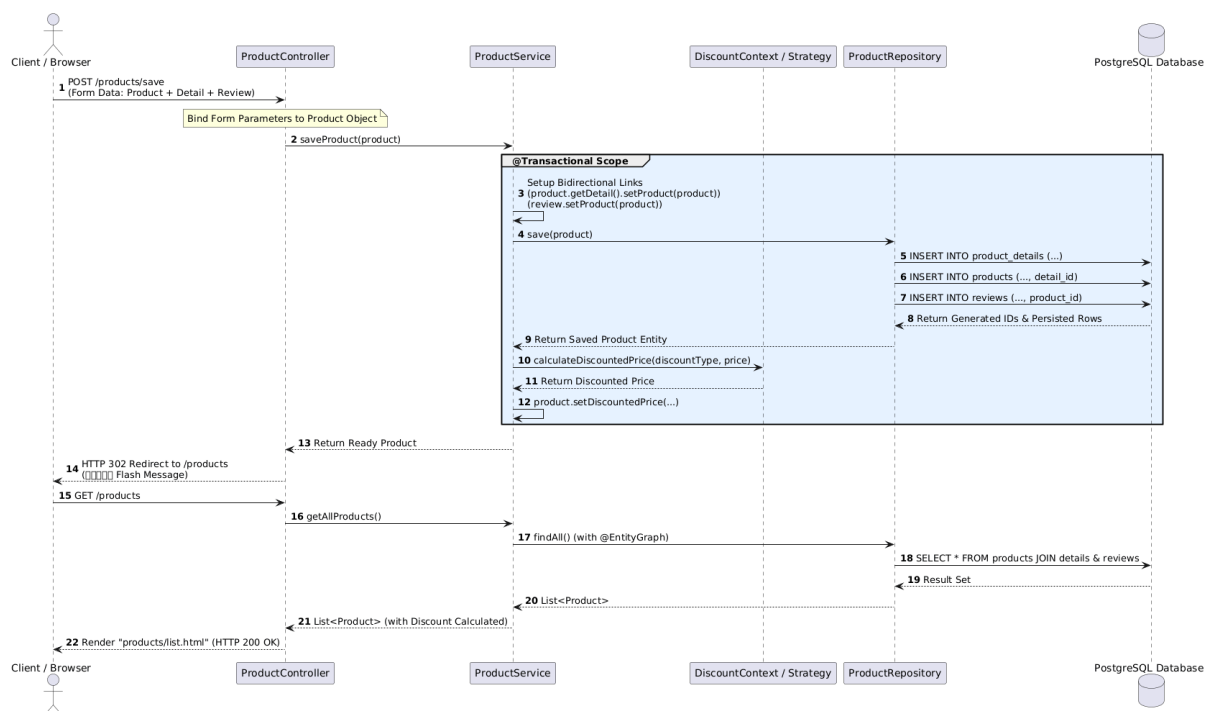
1. Strategy Interface (DiscountStrategy): กำหนด Contract สำหรับทุกกลยุทธ์ ได้แก่ `applyDiscount(double originalPrice)` และ `getDiscountType()`
2. Concrete Strategies:
  - NoDiscountStrategy: คำนวณราคาเดิม ไม่ลดราคา (`originalPrice`)
  - MemberDiscountStrategy: ลด 10% (`originalPrice * 0.90`)

## 673380066-4 นายสัพพัญญู คำตุ้ม

- SeasonalSaleStrategy: ลด 20% ( $\text{originalPrice} * 0.80$ )
3. Context Class (DiscountContext):
- ลงทะเบียนเป็น Spring Component (@Component)
  - ใช้ความสามารถของ Spring Framework ในการรวบรวมคลาสที่ implement DiscountStrategy ทั้งหมดเข้ามาผ่าน Constructor Injection (List<DiscountStrategy>) และจัดเก็บลงใน Map<String, DiscountStrategy>
  - ฟังก์ชัน calculateDiscountedPrice(type, price) จะดึง Strategy ที่ตรงกับประเภทส่วนลดมาประมวลผลทันที โดยไม่ต้องมี if-else

### 1.4 Execution Flow ตั้งแต่ HTTP Request → DB

แผนภาพลำดับขั้นตอน (Sequence Diagram) แสดงเส้นทางการทำงานตั้งแต่ Client ส่ง HTTP Request จนกระทั่งบันทึกลง PostgreSQL Database และส่งผลลัพธ์กลับ:



#### คำอธิบายขั้นตอนการทำงาน

1. HTTP Request Phase: ผู้ใช้งานกรอกฟอร์มเพิ่มสินค้าและส่ง POST /products/save ไปยังเซิร์ฟเวอร์
2. Controller Layer: ProductController รับ Request และทำ Data Binding แปลงข้อมูลจากฟอร์มให้กลายเป็น Object Product ที่มี Object ProductDetail และ Review แแนบอยู่ภายใน
3. Service Layer & Transaction:
  - ProductService ที่มี Annotation @Transactional รับช่วงต่อ
  - ทำการจัดการความสัมพันธ์แบบสองทาง (Bidirectional Relationship Management) โดยเซ็ทให้ Child Object ชี้กลับมาที่ Parent Object
4. Repository & ORM Layer:
  - เรียก productRepository.save(product)

## 673380066-4 นายสัพพัญญู คำตุ้ม

- Spring Data JPA ร่วมกับ Hibernate จะวิเคราะห์ CascadeType.ALL และสร้างคำสั่ง SQL INSERT เรียงลำดับอย่างถูกต้อง: บันทึก product\_details ก่อนเพื่อนำ Primary Key มาใส่เป็น detail\_id (FK) ในตาราง products และบันทึก reviews โดยใส่ product\_id (FK)
- 5. Database Phase: PostgreSQL ตรวจสอบ Constraints (Primary Key, Foreign Key, Not-Null) และบันทึกข้อมูลลงดิสก์
- 6. Post-Processing & Strategy: Service เรียกใช้งาน DiscountContext เพื่อคำนวณราคาหลังหักส่วนลด และเช็คใส่ตัวแปรชั่วคราว (@Transient discountedPrice) เพื่อเตรียมแสดงผล
- 7. HTTP Response Phase: Controller ส่งสัญญาณ Redirect (HTTP 302) ไปยังหน้าแสดงรายการสินค้า (/products) พร้อม Flash Message แจ้งเตือนความสำเร็จ

ส่วนที่ 2: Code และคำอธิบาย (Source Code & Explanation)

2.1 การออกแบบ Entity ทั้ง 3 ตัว และคำอธิบาย JPA Annotations

2.1.1 Product.java

ไฟล์ Entity หลักที่เป็นศูนย์กลางของความสัมพันธ์ทั้งแบบ 1:1 และ 1:N:

```
package com.example.demo.model;

import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false)
    private String category;

    @Column(nullable = false)
    private String brand;

    @Column(nullable = false)
    private Integer stock;

    @Column(nullable = false)
    private Double price;

    @Column(nullable = false)
    private String discountType = "NONE";

    // --- ความสัมพันธ์ 1:1 กับ ProductDetail (Owning Side) ---
    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "detail_id", referencedColumnName = "id")
    private ProductDetail detail;

    // --- ความสัมพันธ์ 1:N กับ Review (Inverse Side) ---
    @OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Review> reviews = new ArrayList<>();

    // ฟิลด์สำหรับคำนวณราคาหลังหักส่วนลด ไม่ต้องบันทึกลง Database
    @Transient
    private Double discountedPrice;

    // Constructors, Getters, Setters, and Helper Methods...
    public void setDetail(ProductDetail detail) {
        this.detail = detail;
        if (detail != null) {
            detail.setProduct(this);
        }
    }
}
```

```

public void addReview(Review review) {
    if (this.reviews == null) {
        this.reviews = new ArrayList<>();
    }
    this.reviews.add(review);
    review.setProduct(this);
}
}

```

### คำอธิบาย Annotations ใน Product.java

- @Entity: ประกาศว่าคลาสนี้เป็น JPA Entity ที่จะถูกแมปเข้ากับตารางในฐานข้อมูล
- @Table(name = "products"): กำหนดชื่อตารางในฐานข้อมูล PostgreSQL ให้เป็น products
- @Id: ระบุว่าฟิลด์ id ทำหน้าที่เป็น Primary Key (PK) ของตาราง
- @GeneratedValue(strategy = GenerationType.IDENTITY): กำหนดให้ Database เป็นผู้สร้างค่า PK อัตโนมัติ โดยใช้คุณสมบัติ auto-increment (SERIAL ใน PostgreSQL)
- @Column(nullable = false): กำหนดข้อจำกัด (Constraint) ให้คอลัมน์ใน DB ต้องไม่เป็นค่าว่าง (NOT NULL)
- @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true): กำหนดความสัมพันธ์แบบ 1:1 กับ ProductDetail; cascade = CascadeType.ALL ทำให้ทุกการเปลี่ยนแปลงของ Product ส่งต่อไปยัง ProductDetail อัตโนมัติ; orphanRemoval = true ทำให้เมื่อตัดการเชื่อมโยงออก อ็อบเจกต์นั้นจะถูกลบออกจากรฐานข้อมูลทันที
- @JoinColumn(name = "detail\_id", referencedColumnName = "id"): ระบุว่าตาราง products เป็น Owning Side ที่ถือคอลัมน์ Foreign Key ชื่อ detail\_id เชื่อมโยงไปยัง PK id ของตาราง product\_details
- @OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true): กำหนดความสัมพันธ์แบบ 1:N กับ Review; mappedBy = "product" บ่งบอกว่าฝั่งนี้เป็น Inverse Side โดยให้ฟิลด์ product ในคลาส Review เป็นตัวควบคุมความสัมพันธ์และเป็นผู้ถือ Foreign Key
- @Transient: แจ้ง JPA ว่าฟิลด์ discountedPrice ใช้สำหรับการคำนวณใน Application เท่านั้น ไม่ต้องสร้างเป็นคอลัมน์ในฐานข้อมูล

### 2.1.2 ProductDetail.java

Entity สำหรับเก็บข้อมูลจำเพาะเชิงลึกของสินค้า (ฝั่ง Inverse Side ของ 1:1):

```

package com.example.demo.model;

import jakarta.persistence.*;

@Entity
@Table(name = "product_details")
public class ProductDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(columnDefinition = "TEXT")
    private String description;

    @Column
    private String warranty;

    @Column

```

```
private Double weight;

@Column
private String dimensions;

@Column
private String manufacturedCountry;

// ฝ่าย Inverse Side ของความสัมพันธ์ 1:1
@OneToOne(mappedBy = "detail")
private Product product;

// Constructors, Getters, Setters...
}
```

### คำอธิบาย Annotations ใน ProductDetail.java

- @Entity และ @Table(name = "product\_details"): ระบุตัวตน Entity และแมปเข้ากับตาราง product\_details
- @Id และ @GeneratedValue: กำหนด Primary Key แบบ Identity
- @Column(columnDefinition = "TEXT"): บังคับให้คอลัมน์ description มีชนิดข้อมูลใน PostgreSQL เป็น TEXT เพื่อให้รองรับข้อความบรรยายรายละเอียดสินค้าที่มีความยาวมากได้
- @Column: แมปตัวแปรเป็นคอลัมน์ตามปกติของฐานข้อมูล
- @OneToOne(mappedBy = "detail"): ระบุว่าเป็นฝั่งรับ (Inverse Side) ของความสัมพันธ์ 1:1 โดยอ้างอิงชื่อฟิลด์ detail ในคลาส Product ทำให้ตาราง product\_details ไม่ต้องมีคอลัมน์ FK ซ้ำซ้อน

### 2.1.3 Review.java

Entity สำหรับจัดเก็บรีวิวและความคิดเห็นของสินค้า (ฝั่ง Owning Side ของ 1:N):

```
package com.example.demo.model;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name = "reviews")
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column
    private String reviewer;

    @Column
    private Integer rating;

    @Column(columnDefinition = "TEXT")
    private String comment;

    @Column
    private LocalDate reviewDate;
}
```

```
// ฝั่ง Many เป็นผู้ถือครอง Foreign Key (product_id)
@Entity
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "product_id")
private Product product;

// Constructors, Getters, Setters...
}
```

#### คำอธิบาย Annotations ใน Review.java

- `@Entity` และ `@Table(name = "reviews")`: แมปคลาสเข้ากับตาราง reviews
- `@Column(columnDefinition = "TEXT")`: คอลัมน์ comment ใช้ชนิดข้อมูล TEXT รองรับข้อความรีวิวขนาดยาว
- `@ManyToOne(fetch = FetchType.LAZY)`: กำหนดความสัมพันธ์แบบ N:1 จาก Review หลายรายการไปยัง Product 1 รายการ; `fetch = FetchType.LAZY` โหลดข้อมูล Product แบบ Lazy Loading เพื่อประสิทธิภาพสูงสุดในการดึงข้อมูลรีวิว
- `@JoinColumn(name = "product_id")`: ระบุว่าตาราง reviews เป็น Owning Side ของความสัมพันธ์ 1:N และมีคอลัมน์ Foreign Key ชื่อ `product_id` ซึ่งไปยัง Primary Key ของตาราง products



## 2.2 Service และ Controller พร้อมคำอธิบาย Constructor Injection

### 2.2.1 ProductService.java

ทำหน้าที่เป็นศูนย์กลางของ Business Logic, Transaction Boundary และการประสานความสัมพันธ์:

```
package com.example.demo.service;

import com.example.demo.model.Product;
import com.example.demo.model.ProductDetail;
import com.example.demo.model.Review;
import com.example.demo.repository.ProductRepository;
import com.example.demo.strategy.DiscountContext;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

@Service
@Transactional
public class ProductService {

    private final ProductRepository productRepository;
    private final DiscountContext discountContext;

    public ProductService(ProductRepository productRepository, DiscountContext discountContext) {
        this.productRepository = productRepository;
        this.discountContext = discountContext;
    }

    @Transactional(readOnly = true)
    public List<Product> getAllProducts() {
        List<Product> products = productRepository.findAll();
        for (Product product : products) {
            calculateAndSetDiscountedPrice(product);
        }
        return products;
    }

    public Product saveProduct(Product product) {
        // จัดการความสัมพันธ์ 1:1 แบบสองทาง
        if (product.getDetail() != null) {
            product.getDetail().setProduct(product);
        }

        // จัดการความสัมพันธ์ 1:N กับ Review
        if (product.getReviews() != null) {
            List<Review> validReviews = new ArrayList<>();
            for (Review review : product.getReviews()) {
                if (review != null && review.getReviewer() != null && !review.getReviewer().trim().isEmpty()) {
                    review.setProduct(product);
                    if (review.getReviewDate() == null) {
                        review.setReviewDate(LocalDate.now());
                    }
                    validReviews.add(review);
                }
            }
            product.setReviews(validReviews);
        }

        Product savedProduct = productRepository.save(product);
```

```

        calculateAndSetDiscountedPrice(savedProduct);
        return savedProduct;
    }

    // Methods updateProduct, deleteProduct, calculateAndSetDiscountedPrice...
}

```

## 2.2.2 ProductController.java

ทำหน้าที่ประสานงานระหว่าง Web Request, Model และ View Template:

```

package com.example.demo.controller;

import com.example.demo.model.Product;
import com.example.demo.model.ProductDetail;
import com.example.demo.model.Review;
import com.example.demo.service.ProductService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping("/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public String listProducts(Model model) {
        model.addAttribute("products", productService.getAllProducts());
        return "products/list";
    }

    @GetMapping("/add")
    public String showAddForm(Model model) {
        Product product = new Product();
        product.setDetail(new ProductDetail());
        product.getReviews().add(new Review()); // เตรียมไว้สำหรับฟอร์ม reviews[0]
        model.addAttribute("product", product);
        return "products/add";
    }

    @PostMapping("/save")
    public String saveProduct(@ModelAttribute Product product, RedirectAttributes redirectAttributes) {
        productService.saveProduct(product);
        redirectAttributes.addFlashAttribute("message", "บันทึกข้อมูลสินค้าเรียบร้อยแล้ว");
        return "redirect:/products";
    }

    // Handlers อื่นๆ: edit, update, delete...
}

```

## 2.2.3 เหตุผลและความสำคัญของ Constructor Injection

## 673380066-4 นายสัพพัญญู คำตุ้ม

ในโปรเจกต์นี้เลือกใช้ Constructor Injection แทนการใช้ Field Injection ด้วย @Autowired บนตัวแปรโดยตรง ด้วยเหตุผลสำคัญ 5 ประการ:

1. ความไม่เปลี่ยนแปลง (Immutability): สามารถกำหนดตัวแปร dependencies เป็น final (เช่น private final ProductRepository productRepository;) ได้ ทำให้มั่นใจได้ว่า Dependencies จะถูกกำหนดค่าตั้งแต่ตอนสร้าง Object เพียงครั้งเดียว และไม่สามารถถูกแทนที่หรือเปลี่ยนแปลงได้ระหว่างการทำงาน
2. การรับประกันความสมบูรณ์ของ Object (Null-Safety): หากไม่มี Dependencies ที่จำเป็น Object จะไม่สามารถถูก Instantiate ได้ ป้องกันปัญหา NullPointerException ในขณะ Runtime ได้อย่างสมบูรณ์
3. ความง่ายในการทดสอบ (Testability): สามารถเขียน Unit Test ได้ง่ายโดยไม่ต้องพึ่งพา Spring Context หรือ Reflection เพียงสร้าง Mock Object (เช่น Mockito) แล้วส่งผ่าน Constructor ตรง ๆ:

```
ProductService service = new ProductService(mockRepo, mockContext);
```

4. ป้องกันปัญหา Circular Dependency (ตรวจจับได้ทันที): หากมีการเรียกฟังก์ชันกันเป็นวงกลม Spring IoC Container จะตรวจพบตั้งแต่ Application Startup และแสดงข้อผิดพลาดทันที (Fail-Fast)
5. ความชัดเจนของ Dependencies ตามหลัก DIP & Clean Code: คลาสใดต้องการอะไรเพื่อทำงาน จะถูกประกาศอย่างเปิดเผยผ่าน Signature ของ Constructor ทำให้ผู้พัฒนาเข้าใจการทำงานของคลาสได้ทันที

673380066-4 นายสัพพัญญู คำตุ้ม

### ส่วนที่ 3: ภาพหน้าจอการทำงาน (Application Screenshots)

หมายเหตุสำคัญ: ตามเกณฑ์ของ Lab 8 ชื่อสินค้าในทุกรูปภาพต้องระบุรหัสนักศึกษา และ Section เช่น iPhone 15 Pro (673380123-4 SEC 1)

#### 3.1 หน้าเพิ่มสินค้า (Create)

The screenshot shows a web browser window with the URL `http://localhost:8080/products/add`. The page title is '+ เพิ่มสินค้าใหม่'. The form contains the following fields:

- ข้อมูลสินค้า (Product):**
  - ชื่อสินค้า: iPhone 15 Pro (673380066-4 SEC 1)
  - หมวดหมู่: Phone
  - ยี่ห้อ: Apple
  - จำนวนสินค้า: 1000
  - ราคาต่อหน่วย (บาท): 500
  - ประเภทส่วนลด: ส่วนลดคงที่ (20%)
- ข้อมูลเสริมสินค้า (ProductDetail) - ความสัมพันธ์ 1:1:**
  - จากชื่อสินค้า: จอสูง 6.5 นิ้ว
  - ระยะเวลาประกัน: 1 year
  - น้ำหนัก (กิโลกรัม): 1.67
  - ขนาด (กว้าง x สูง): 150.7 x 71.4 มม.
  - ประเทศต้นกำเนิด: Thailand
- รีวิวสินค้า (Review) - ความสัมพันธ์ 1:N:**
  - ชื่อผู้รีวิว: admin
  - คะแนน (1-5): 5
  - คำติชม: ดีมาก

#### 3.2 หน้ารายการสินค้า (Read)

The screenshot shows a web browser window with the URL `http://localhost:8080/products`. The page title is 'Product Shop'. Below the title, there is a green bar with the text 'สินค้าที่เพิ่มเข้ามาในระบบแล้ว' and a blue button '+ เพิ่มสินค้าใหม่'. The main content is a table with the following data:

| # | ชื่อสินค้า                        | หมวดหมู่ | ยี่ห้อ | สต็อก | ราคาต่อหน่วย | ส่วนลด   | ราคาต่อหน่วย (หลังส่วนลด) | ระยะเวลาประกัน | น้ำหนัก (kg) | รีวิว   | สถานะ    |
|---|-----------------------------------|----------|--------|-------|--------------|----------|---------------------------|----------------|--------------|---------|----------|
| 1 | iPhone 15 Pro (673380066-4 SEC 1) | Phone    | Apple  | 1000  | 500.00       | SEASONAL | 400.00                    | 1 year         | 1.67         | 1 รีวิว | มีสินค้า |
| 2 | iPhone 20 Pro (673380066-4 SEC 1) | Phone    | Apple  | 200   | 1000.00      | MEMBER   | 900.00                    | 1 year         | 2.65         | 0 รีวิว | มีสินค้า |

#### 3.3 หน้าแก้ไขสินค้า (Update)

673380066-4 นายสัพพัญญู คำตุ้ม

Product Shop Lab 8 - 11 & 12 Relationship

ข้อมูลทั้งหมดนี้เป็นข้อมูลสมมติ

+ เพิ่มสินค้าใหม่

| # | ชื่อสินค้า                        | ประเภท | ยี่ห้อ | รุ่น | ราคา    | จำนวน    | ประเภท | ระยะเวลา | ราคาต่อหน่วย | สถานะ |    |
|---|-----------------------------------|--------|--------|------|---------|----------|--------|----------|--------------|-------|----|
| 1 | iPhone 16 Pro (673380066-4 SEC 1) | Phone  | Apple  | 1000 | 500.00  | SEASONAL | 400.00 | 1 year   | 1.67         | 11%   | ลบ |
| 2 | iPhone 20 Pro (673380066-4 SEC 1) | Phone  | Apple  | 200  | 1000.00 | MEMBER   | 900.00 | 1 year   | 2.65         | 0.9%  | ลบ |

### 3.4 หน້ายืนยันและผลการลบสินค้า (Delete)

673380066-4 นายสัพพัญญู คำตุ้ม

ยืนยันการลบสินค้า

คุณต้องการลบสินค้านี้หรือไม่? การดำเนินการนี้ไม่สามารถย้อนกลับได้

|             |                                   |
|-------------|-----------------------------------|
| ชื่อสินค้า: | iPhone 20 Pro (873380066-4 SEC 1) |
| หมวดหมู่:   | Phone                             |
| ยี่ห้อ:     | Apple                             |
| ราคา:       | 1000.00 บาท                       |
| ระยะเวลา:   |                                   |
| ประกัน:     | 1 year                            |

⚠️ ข้อมูลสินค้า (Product Data) จะถูกลบทิ้งไปอย่างถาวร (เนื่องจาก CascadeType ALL)

**ยืนยัน** **ยกเลิก**

Product Shop Lab 8 - IT & VR Relationship

ลบสินค้าที่ยกเลิกแล้ว

• เพิ่มสินค้าใหม่

| # | ชื่อสินค้า                        | หมวดหมู่ | ยี่ห้อ | ราคา | สต็อก  | ราคาพิเศษ | ระยะเวลา (ปี) | จำนวน (ปี) | วันที่ | จำนวน |
|---|-----------------------------------|----------|--------|------|--------|-----------|---------------|------------|--------|-------|
| 1 | iPhone 16 Pro (873380066-4 SEC 1) | Phone    | Apple  | 1000 | 500.00 | SEASONAL  | 400.00        | 1 year     | 1,67   | 1 151 |

### 3.5 โครงสร้างฐานข้อมูลใน pgAdmin (Database & Foreign Keys)

The image displays two screenshots of the pgAdmin 4 interface, showing database queries and their results for a PostgreSQL 18 database named 'lab8shop'.

**Top Screenshot:**

- Object Explorer:** The 'reviews' table is selected under the 'products' schema.
- Query:** `select * from reviews`
- Data Output:**

| id [PK] bigint | comment text | rating integer | review_date date | reviewer character varying (255) | product_id bigint |
|----------------|--------------|----------------|------------------|----------------------------------|-------------------|
| 1              | 1            | 5              | 2026-09-09       | สัพ นล้า                         | 1                 |
- Status:** Total rows: 1, Query complete 00:00:00.094

**Bottom Screenshot:**

- Object Explorer:** The 'products' table is selected under the 'products' schema.
- Query:** `select * from products`
- Data Output:**

| id [PK] bigint | brand character varying (255) | category character varying (255) | discount_type character varying (255) | name character varying (255)      | price double precision | stock integer | detail_id bigint |
|----------------|-------------------------------|----------------------------------|---------------------------------------|-----------------------------------|------------------------|---------------|------------------|
| 1              | Apple                         | Phonk                            | SEASONAL                              | iPhone 16 Pro (673380066-4 SEC... | 500                    | 1000          | 1                |
- Status:** Total rows: 1, Query complete 00:00:00.098

673380066-4 นายสัพพัญญู คำต๋ม

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Materialized Views
- Operators
- Procedures
- Sequences
- Tables (3)
  - product\_details
    - Columns
    - Constraints
    - Indexes
    - RLS Policies
    - Rules
    - Triggers
  - products
    - Columns
    - Constraints
    - Indexes
    - RLS Policies
    - Rules
    - Triggers
  - reviews
    - Columns (6)
    - Constraints
    - Indexes
    - RLS Policies
    - Rules
    - Triggers
  - Trigger Functions
  - Types
  - Views
  - Subscriptions

lab8shop/postgres@PostgreSQL 18\*

lab8shop/postgres@PostgreSQL 18

Query Query History

```
1 select * from product_details
```

Scratch Pad

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

| id          | description | dimensions              | manufactured_country    | warranty                | weight           |
|-------------|-------------|-------------------------|-------------------------|-------------------------|------------------|
| [PK] bigint | text        | character varying (255) | character varying (255) | character varying (255) | double precision |
| 1           | ของเล่นเด็ก | 15x7x1cm                | Thailand                | 1 year                  | 1.67             |

Total rows: 1 Query complete 00:00:00.207 CRLF Ln 1, Col 30